

Linux & Git 보고서

로빛 예비단원 주호진

Linux

리눅스란?

Linux는 Windows나 MacOS처럼 하나의 운영체제이다. Windows는 MS 직원만, MacOS는 애플 직원만이 개발, 수정하지만, Linux는 무료 오픈 소스이기 때문에 누구나 개발, 수정할 수 있다. 그렇기에 수많은 사람들이 Linux에 기여하고 있다.

왜 리눅스인가?

사실 윈도우나 MacOS를 쓰던 사람에게 가장 먼저 드는 생각이 아닐까싶다. 리눅스가 또 하나의 운영체제인 건 알겠는데, 그래서 리눅스를 쓰는 이유가 무엇인지가 궁금할 것이다.

1. 무료

리눅스는 무료이기 때문이다. 윈도우는 유료이고, MacOS는 사용하려면 애플 장비를 구매해야하는 재정적 부담이 존재한다. 그에 반해 리눅스는 장비도, 사용료도 없다. 따라서 기업의 경우 비용 절감을 위해서도 리눅스를 사용하는 경우가 많다.

2. 오픈 소스

오픈 소스이기 때문에 누구나 수정, 배포에 참여할 수 있으며, 기여하는 사람이 많은만큼 다양한 목적의 배포판들이 존재한다. 따라서 본인이 사용하고자 하는 목적에 알맞는 배포판을 선택해서 사용할 수도 있다.

3. 유지보수

오픈 소스인만큼 다양한 개발자들이 문제들을 끊임없이 개선하기 때문에 유지

보수가 꾸준히 진행된다는 장점이 있다.

4. 자유도

누구나 수정할 수 있으며, 그에 따라 자신의 목적에 맞는 배포판을 만들어낼 수 있다. 위에서 언급했지만 사용하고자 하는 목적에 맞게 배포판들을 사용할 수도 있다. 앞으로 사용하게 될 Ubuntu도 리눅스의 수많은 배포판 중에 하나이다.

위에 기재한 내용 외에도 안정성, 보안, 이식성 등 리눅스 환경에서의 장점이 많다. 특히 현재 로봇 관련한 환경들은 대부분이 Linux 기반에서 호환이 더 자유롭기 때문에 특정 분야에서는 linux 기반 환경이 필수라고 볼 수 있고, 아이폰을 제외하고 범용적으로 사용되는 Android도 리눅스 커널을 기반으로 제작되었다.

리눅스 명령어

▼ 다루는 명령어 요약표

01 pwd	02 ls	03 cd	04 mkdir
현재 위치 출력	현재 디렉터리 내의 파일과 디렉터리 출력	디렉터리 이동	디렉터리 생성
05 cp	06 mv	07 rm	08 cat
파일 또는 디렉터리 복사	파일 또는 디렉터리 이동	파일 또는 디렉터리 삭제	파일 내용을 확인
09 touch	10 echo	11 ip addr/ifconfig	12 ss
빈 파일을 생성	문자열 화면에 표시	IP 정보 확인	네트워크 상태 확인
13 nc	14 which, whereis, locate	15 tail	16 find
서버의 포트 확인	명령어 위치 확인	파일의 마지막 부분 확인하기	파일이나 디렉터리 찾기
17 ps	18 grep	19 kill	20 alias
현재 실행 중인 프로세스 목록과 상태 확인	주어진 입력값에서 패턴에 맞는 값 출력	프로세스 종료	명령어 별칭 만들기
21 vi/vim	편집기		

위 사진은 자주쓰는 리눅스 명령어이다. 리눅스 명령어가 꽤 많기 때문에 모든 명령어를 숙지하고 사용하는 것보다는 자주 사용하는 명령어를 숙지하고 사용하다가, 필요한

명령어가 있을 때마다 추가적으로 검색해서 사용하는 것이 좋아보인다.

리눅스 배포판 종류

리눅스 배포판은 크게 3가지로 나뉜다. RedHat 계열, Debian 계열, Slacware 계열이 있다.

레드햇 계열은 기업 환경에 특화된 리눅스이고, 데비안 계열은 다양한 목적에 맞게 특화된 리눅스가 특징이다. Ubuntu 또한 데비안 계열의 리눅스 중 하나이고 우분투의 특징은 사용하기 쉬운 인터페이스와, 광범위한 하드웨어 지원이다. 슬랙웨어 계열은 단순함이 특징인 리눅스 배포판이다.

리눅스의 파일 시스템

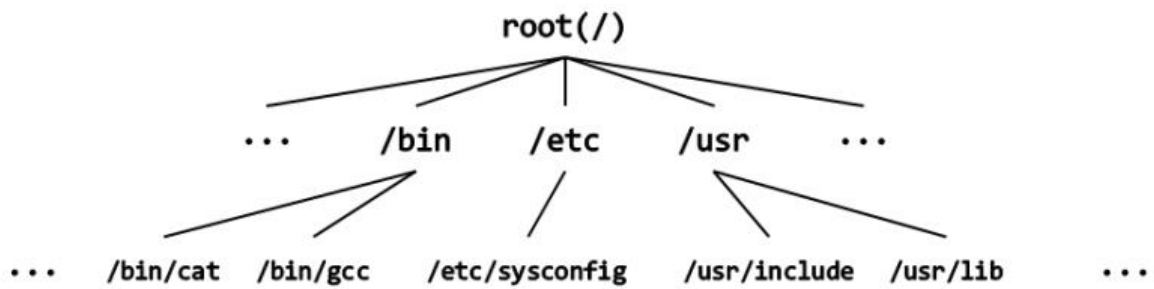
파일 시스템은 컴퓨터의 저장장치에 데이터를 저장하고 관리하는 체계를 의미한다. 리눅스 기반 운영체제는 FHS(Filesystem Hierarchy Standard)라는 규칙을 따른다. 리눅스를 사용하려면 리눅스 파일 시스템 규칙에 따라 데이터를 저장하고 관리해야 하는데, 그렇기 때문에 FHS에 대한 개념을 알고 있으면 좋다.

Everything is a file

리눅스는 모든 것을 파일로 취급한다. 텍스트, 이미지, 오디오같은 데이터부터 키보드, 마우스, 모니터, RAM 등의 장치들도 리눅스에서는 file처럼 취급한다. 이러한 이유로 리눅스의 파일 시스템은 통일성과 유연성이 좋다.

루트 파일 시스템

리눅스 파일 시스템을 루트 파일 시스템이라고 부르기도 하는데, 이는 리눅스가 단일 계층 트리 구조이기 때문이다. Root를 제외한 모든 파일과 디렉터리와 장치는 root 디렉터리 아래에 존재한다. 이러한 구조는 직관적이고 통일성이 있다는 장점을 가지게 된다. 최상위 디렉토리인 root는 / 으로 표현한다.



권한

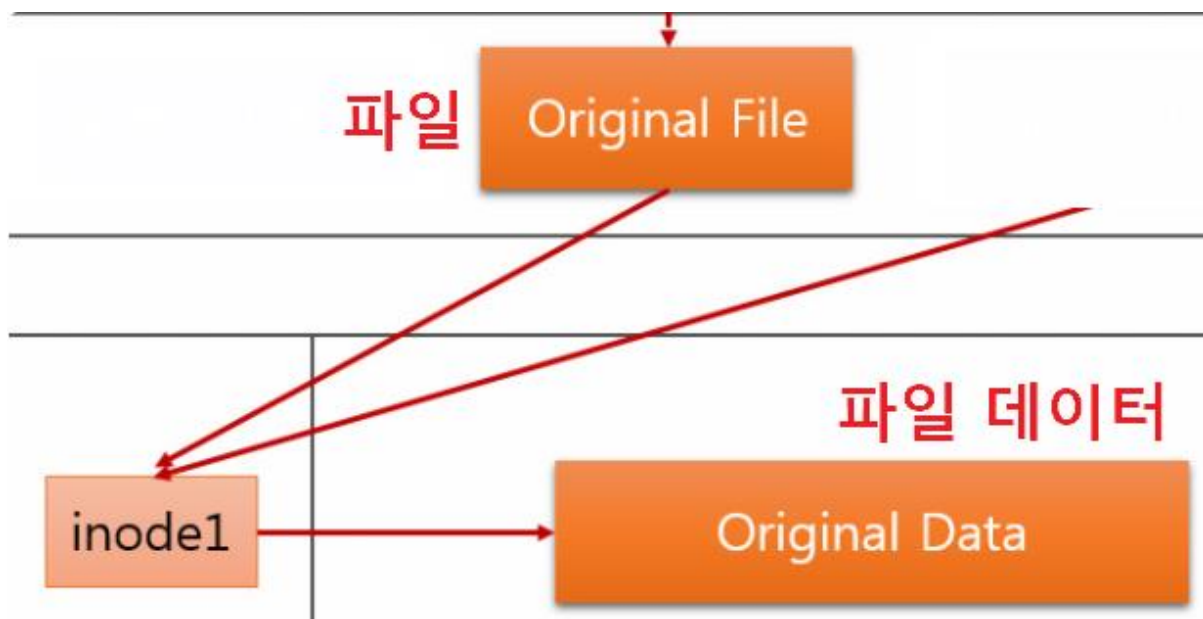
리눅스는 3가지의 권한 범주와 3가지 권한이 있다. 권한 범주는 소유자(Owner), 그룹(Groups), 기타(Others)로 구분되고, 권한은 읽기(Read), 쓰기(Write), 실행(execute)가 있다. 모든 파일들은 3가지 범주와 3가지 권한에 대한 정보를 가지고 있고, 리눅스는 모든 것을 파일로 다루기 때문에 모든 정보에 대해 범주와 권한에 대한 정보를 알 수 있다. 이렇게 파일 속성에 관한 것을 메타데이터(metadata)라고 부른다.

파일 확장자

리눅스에서는 파일 확장자라는 개념이 없다. 사용자가 파일의 종류를 식별하기 위해 붙이는 것일 뿐이다. 리눅스는 파일들이 가지고 있는 메타데이터 등의 정보를 가지고 종류를 식별하게 된다.

링크

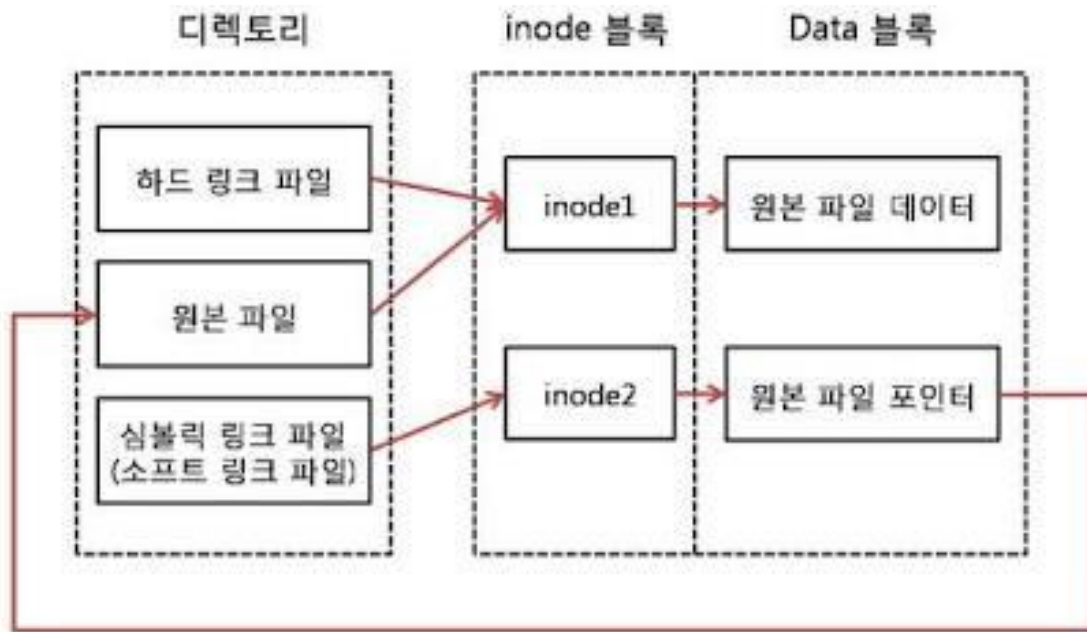
윈도우로 어떤 파일을 실행할 때 바로가기를 사용해본 적이 있을 것이다. 리눅스도 윈도우 바로가기와 같은 비슷한 기능을 제공하는데, 이를 링크라고 한다. 리눅스에서 링크는 하드 링크와 심볼릭 링크(소프트 링크라고도 한다)로 나뉘어져 있다.



링크에 대해 설명하기 전에 먼저 inode라는 개념이 있다. Inode는 파일의 허가권, 소유권, 실제 위치 등의 정보가 담겨져 있다. 실제 파일의 내용을 가지고 있는 것은 아니고, 정확히는 파일의 내용을 가지고 있는 주소를 가진다.

하드 링크는 원본 파일과 동일한 inode를 직접적으로 가리킨다. 따라서 원본 파일이 사라지더라도 데이터가 살아있다면 원본 파일에 접근이 가능하다. 이와 같은 특성 때문에 하드 링크는 데이터를 안전하게 관리하고자 할 때 주로 사용된다.

심볼릭 링크(소프트 링크)는 또 다른 inode를 생성해서 이를 가리킨다. 생성된 inode는 원본 파일을 가리키기 때문에 원본 파일이 사라진다면 해당 데이터에 접근할 수 없게 된다.

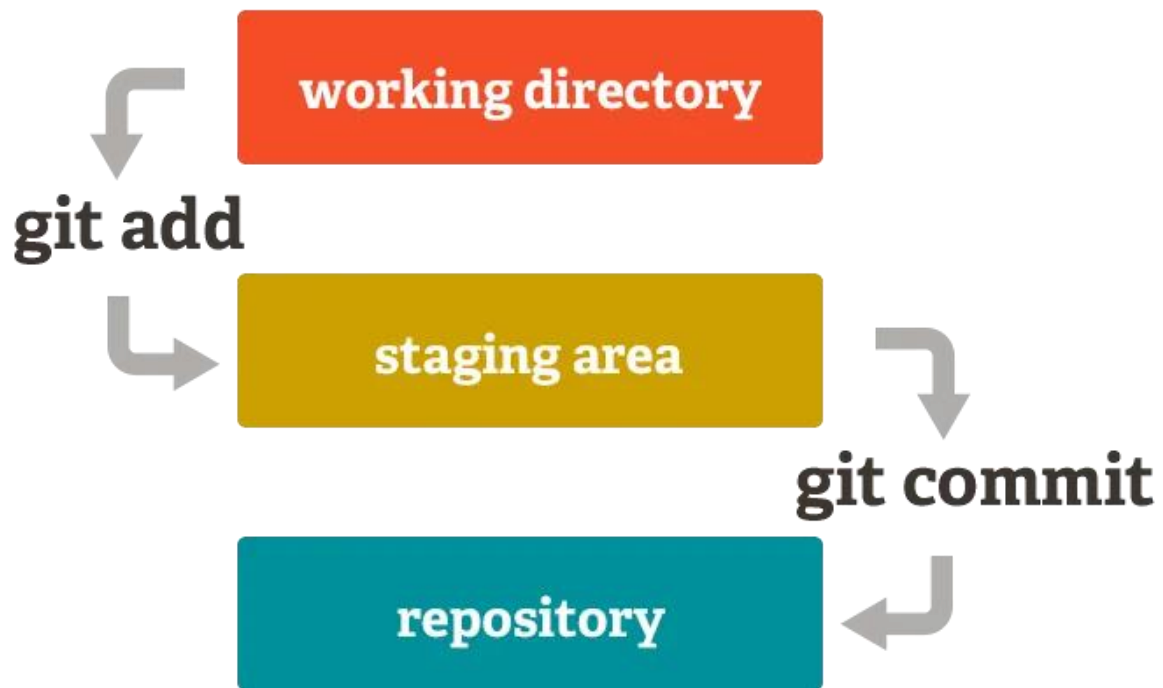


Git

Git이란?

Git은 속도를 중요시하는 버전 관리 시스템이다. 흔히 과제를 할 때 보고서를 쓰거나 발표자료를 만들면 ..._최종, 최최종, 최최최종, 진짜최종 이런 식으로 가는 경우가 한번씩은 있을 것이다. 이러한 파일들을 Git을 이용하면 복사, 백업, 저장을 통해 버전으로 관리할 수 있다.

이러한 버전 관리는 개인이 사용할 때도 이점이 있지만, 여러 명이 동시에 개발을 할 때 장점이 도드러진다고 볼 수 있다. 어떤 특정 분기에서 여러 명이 동시에 다른 작업을 하고 해당 작업들을 나중에 합칠 수 있기 때문이다.



Git은 크게 3단계의 아키텍처를 가지고 있다. working directory는 사용자가 코드를 직접 수정, 추가하는 공간이고 staging area는 repository로 가기 전에 working directory에서 한 작업들을 정리하는 공간이다. Git add를 통해 staging area에 작업들을 등록한 뒤 git commit을 하면 repository에 저장되는 식이다.

Git을 사용할 때 가장 처음 치는 git init을 입력하면 .git 파일이 생기게 되는데 이 .git 파일 내에서 해당 아키텍처가 작동한다.

객체	역할	특징
Blob	파일의 원본 데이터를 저장	내용만 저장, 이름 없음
Tree	디렉터리 구조(폴더) 구조 저장	파일과 디렉터리 정보 포함
Commit	특정 시점의 저장소 상태 기록	부모 Commit 정보 포함
Tag	특정 Commit을 가리키는 참조	주로 버전 관리에 사용

Git에는 blob, tree, commit, tag 4가지의 오브젝트 타입이 있다.

Blob

add가 수행되는 시점의 파일의 내용을 저장한다.

Tree

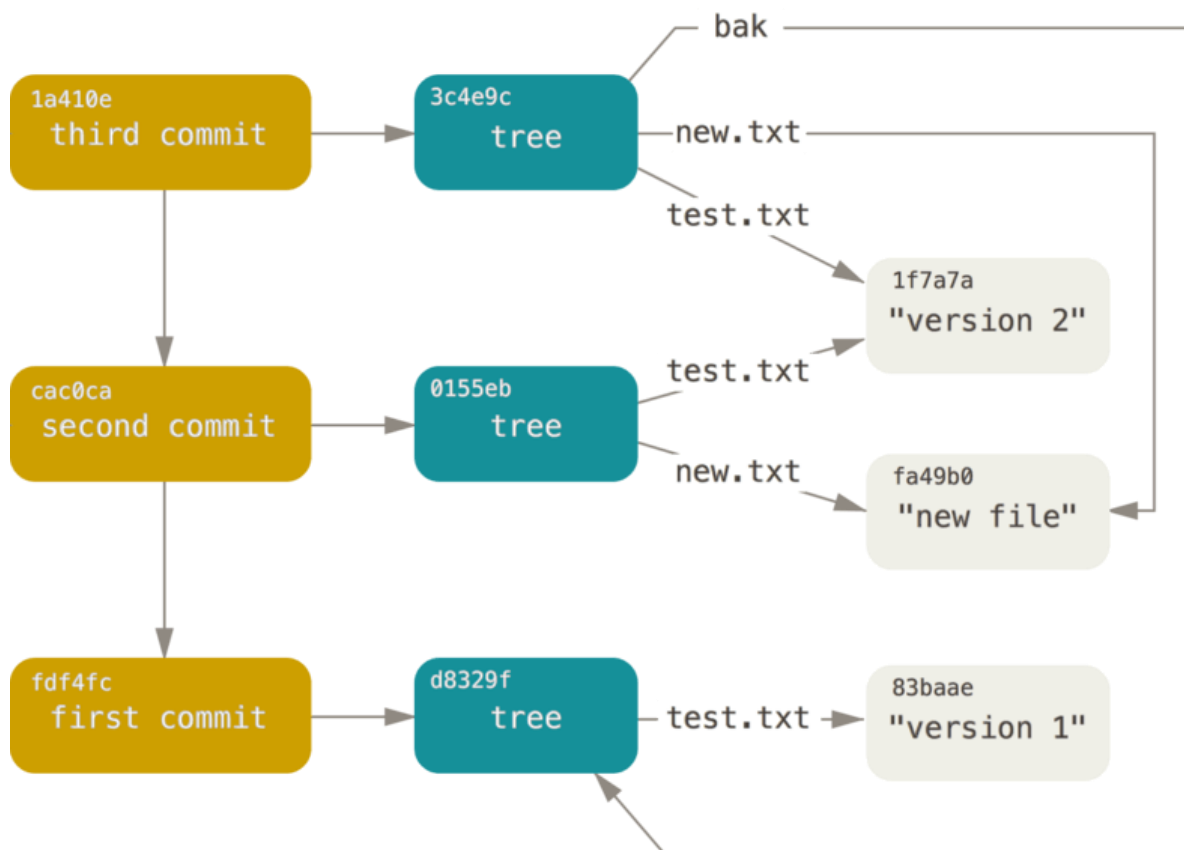
커밋 시점의 디렉터리와 파일 상태를 저장한다.

Commit

변경 사항에 대한 정보(부모 커밋, 작성자, 실행자, 메세지 등)와 커밋 시점에 생성된 tree의 id를 저장한다.

Tag

영어 뜻 그대로 무언가 표시를 위한 태그 기능을 한다. 수많은 커밋 중에 중요한 커밋을 표시할 때 사용하게 되고, 주로 버전 관리에 사용하게 된다.



위 사진을 예시로 보면 test.txt에 "version 1"이라는 text를 저장하여 처음 커밋을 하였다. Blob에는 "version 1"이라는 내용을 가지고 있고, tree는 test.txt가 83baae라는 blob을 가지고 있다는 정보를 저장하게 된다. 첫번째 커밋(fdf4fc)은 해당 시점의 tree(d8329f)를 가리키고, 처음 커밋이므로 부모 커밋은 없게 된다.

두번째에는 new.txt에 "new file"이라는 내용, 그리고 test.txt가 "version 2"로 바뀌게 되

었다. Blob은 "version 2"와 "new file"을 저장하고 tree는 test.txt가 1f7a7a의 blob을 가리키고, new.txt가 fa49b0의 blob을 가리키고 있다는 정보를 가진다. 두번째 커밋(cac0ca)는 첫번째 커밋(fdf4fc)를 부모 커밋으로 가리키고, 해당 시점의 tree(0155eb)를 가리킨다.

세번째에는 bak이라는 새 디렉토리 추가되었다. 해당 디렉토리에는 "version 1"이 내용으로 담긴 test.txt가 담겨 있다. Blob은 여전히 같은 값을 저장하고 있고 tree에서도 기존 test.txt와 new.txt가 바뀐 게 없으므로 기존 blob을 가리킨다. bak에 있는 파일도 첫번째 커밋 시점에서의 tree 구조를 그대로 따라가기 때문에 새로 저장하지 않고 기존에 있던 첫번째 커밋의 tree를 참조하게 된다.

이렇게 기존의 내용이나 tree와 동일한 내용이 생겼을 때 그 값을 새로 저장하지 않고 기존에 저장해둔 blob이나 tree를 참조하므로 속도 측면에서 유리하다.

Git과 Github

Github는 git의 repository의 기능을 특별한 저장소 없이 원격(remote)로 사용할 수 있는 일종의 플랫폼이다. 클라우드를 통해 repository를 관리할 수 있으며, 원격으로 관리하는 코드를 공개하는 대신에 무료로 사용할 수 있다.(최근에는 개인 기준으로 private 저장소도 무료로 사용할 수 있다.) Github가 개인 저장장치를 대신해서 repository를 클라우드에 저장해주는 역할을 한다고 볼 수 있다.

Git 명령어

Git을 사용하기 위해서는 명령어를 알아야 하는데 명령어가 꽤 많다. 따라서 자주 사용하는 명령어를 몸에 익혀 사용하다가, 필요한 기능이 있을 때마다 추가로 명령어를 찾아보면서 사용하는 것이 좋아보인다. 아래는 자주 쓰는 Git 명령어이다.

명령	설명
<code>git init</code>	프로젝트 버전 관리 시작
<code>git remote</code>	원격 저장소 목록 확인
<code>git remote -v</code>	원격 저장소 URL 확인
<code>git remote add <원격별칭> <URL></code>	원격 저장소 추가
<code>git clone <URL></code>	원격 저장소 복제

<code>git status</code>	현재 브랜치의 변경사항 확인
<code>git add <파일></code>	특정 파일 추적 및 스테이징

<code>git commit -m '<메시지>'</code>	버전 생성 (따옴표 닫기 전에는 메시지 줄바꿈 가능)
<code>git log</code>	현재 브랜치의 버전 내역을 확인
<code>git branch</code>	로컬 브랜치 목록 확인
<code>git branch <브랜치></code>	브랜치 생성
<code>git branch -D <브랜치></code>	브랜치 삭제
<code>git checkout <브랜치></code>	브랜치 전환
<code>git push <원격별칭> <브랜치></code>	원격 저장소로 밀어내기

<code>git pull <원격별칭> <브랜치></code>	원격 저장소에서 브랜치 당겨오기
--	-------------------

<code>git fetch</code>	현재 원격 저장소의 브랜치 목록 가져오기
<code>git fetch <원격별칭></code>	특정 원격 저장소의 브랜치 목록 가져오기

<code>git checkout HEAD -- <파일></code>	특정 파일 롤백
<code>git restore <파일></code>	특정 파일 롤백 (v2.23)

<code>git merge <브랜치></code>	현재 브랜치에 특정 브랜치 병합
------------------------------------	-------------------

Git 컨벤션(+깃모지)

앞서 Git은 여러 명과 협업하면서 작업할 때 그 진가를 발휘한다고 언급했다. 하지만 커밋이 쌓일수록, 더 많은 사람이 함께할수록 확인해야 할 코드들이 많아지게 된다. 따라서 프로젝트에 따라 커밋을 할 때 코드에서 어떤 부분이 변경되었는지 명확하게 파악할 수 있도록 하나의 규칙을 세워두고 커밋을 하는 것을 git 컨벤션이라고 한다. 프로젝트 설정마다 조금씩 다르지만, type: subject는 매우 보편적으로 많이 쓰인다.

Type	설명
Feat	새로운 기능 추가
Fix	버그 수정
Docs	문서 수정
Style	스타일 관련 기능 (코드 포매팅, 세미콜론 누락 등)
Refactor	코드 리팩토링
Test	테스트 코드 추가
Chore	빌드 업무 수정, 패키지 매니저 수정

보편적으로 사용되는 type은 위의 사진과 같으며 이외에 프로젝트에서 자주 commit되는 type이 있다면 추가해서 사용한다.

+깃모지

가독성을 위해 깃 컨벤션을 쓰고, 추가적으로 한눈에 대략적인 커밋 내용을 알 수 있도록 이모지를 사용하는 경우도 있다. 깃 + 이모지를 합쳐 깃모지라고 부른다. 깃모지도 깃 컨벤션처럼 규칙을 정해놓고 특정 이모지를 사용한다. [깃모지 사이트](#)에 들어가면 보편적인 경우에서 어떤 상황에 어떤 이모지를 쓰는지에 대한 예가 적혀있다. 물론 프로젝트에 따라 이모지의 의미를 바꿔서 사용할 수도 있다.



:art:

Improve structure /
format of the code.



:zap:

Improve
performance.



:fire:

Remove code or
files.



:bug:

Fix a bug.



:ambulance:

Critical hotfix.



:sparkles:

Introduce new
features.



:memo:

Add or update
documentation.



:rocket:

Deploy stuff.



:lipstick:

Add or update the
UI and style files.



:tada:

Begin a project.



**:white_check_
mark:**

Add, update, or pass
tests.



:lock:

Fix security or
privacy issues.